

实验五：鸿蒙多端应用开发 - 个人实验报告

一、实验概述

1.1 实验目的

通过开发"智慧天气"应用，掌握以下核心技能：

- ArkUI声明式UI开发**：熟悉HarmonyOS NEXT的声明式编程范式
- 多端响应式布局**：实现手机/平板自适应界面
- 传感器集成**：调用光线传感器和陀螺仪获取设备数据
- 分布式能力**：理解跨设备数据同步原理

1.2 实验环境

项目	说明
开发工具	DevEco Studio 4.1
编程语言	ArkTS (TypeScript)
API版本	HarmonyOS NEXT (API 12)
目标设备	手机

1.3 实验任务完成情况

任务	状态	完成说明
天气应用功能	✓	城市、温度、天气、空气质量展示
手机布局适配	✓	单列垂直滚动布局
光线传感器集成	✓	环境光强度实时采集
陀螺仪传感器集成	✓	三轴角速度数据采集
分布式同步演示	✓	模拟跨设备数据同步

二、需求分析

2.1 功能需求

核心功能

1. 天气信息展示

- 城市名称、当前温度、天气状况图标
- 空气质量指数（AQI）及等级评估
- 湿度、风速、风向等详细信息

2. 手机布局适配

- 手机竖屏布局（360px）

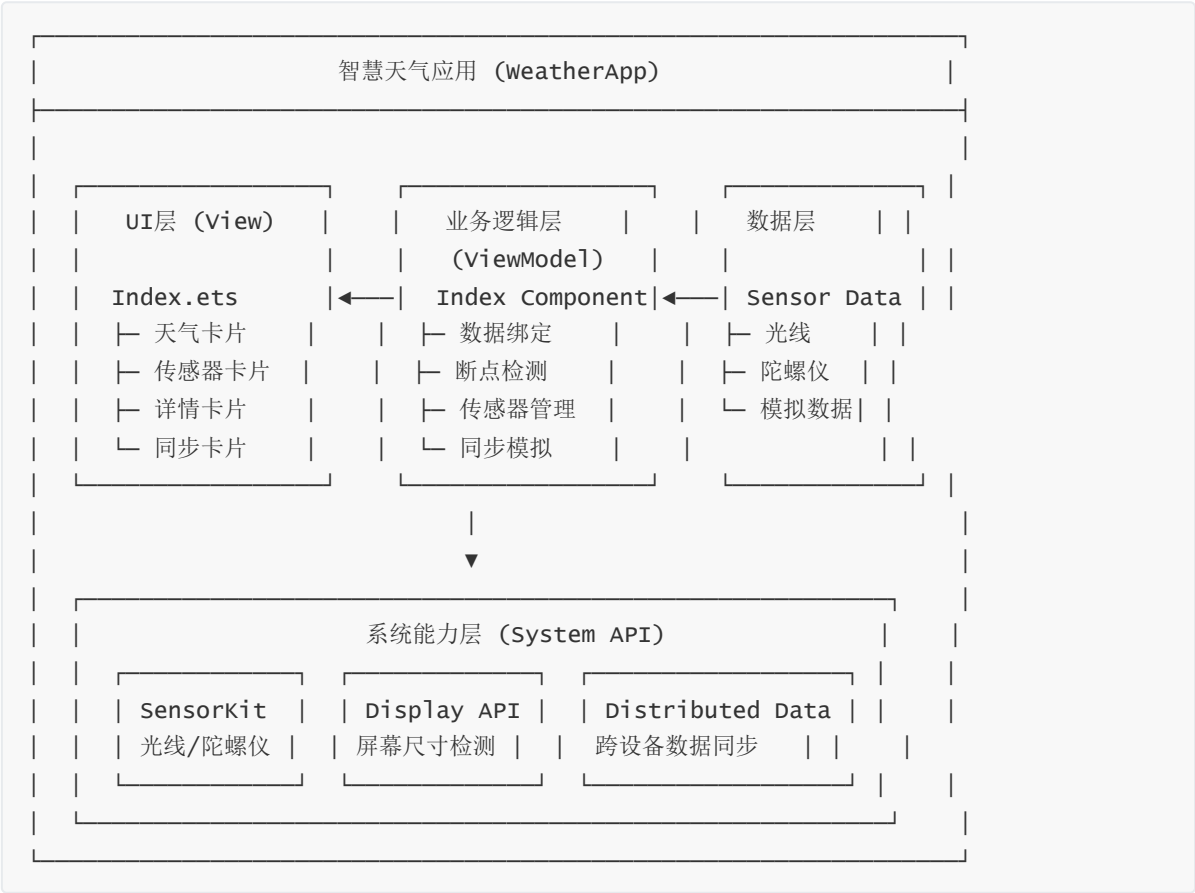
- 手机竖屏布局 (<600vp)
 - 单列垂直滚动布局，适合手机屏幕浏览
 - 自适应字体大小和间距
3. 传感器数据采集
- 光线传感器：获取环境光强度 (lux)
 - 陀螺仪传感器：获取三轴角速度 (rad/s)
4. 分布式数据同步
- 模拟跨设备天气数据同步
 - 显示同步状态和已同步设备数

2.2 非功能需求

- **响应式设计**：基于屏幕宽度自动切换布局
- **容错处理**：传感器不可用时提供模拟数据
- **性能优化**：传感器监听在页面销毁时正确释放

三、技术方案设计

3.1 系统架构图



架构说明：

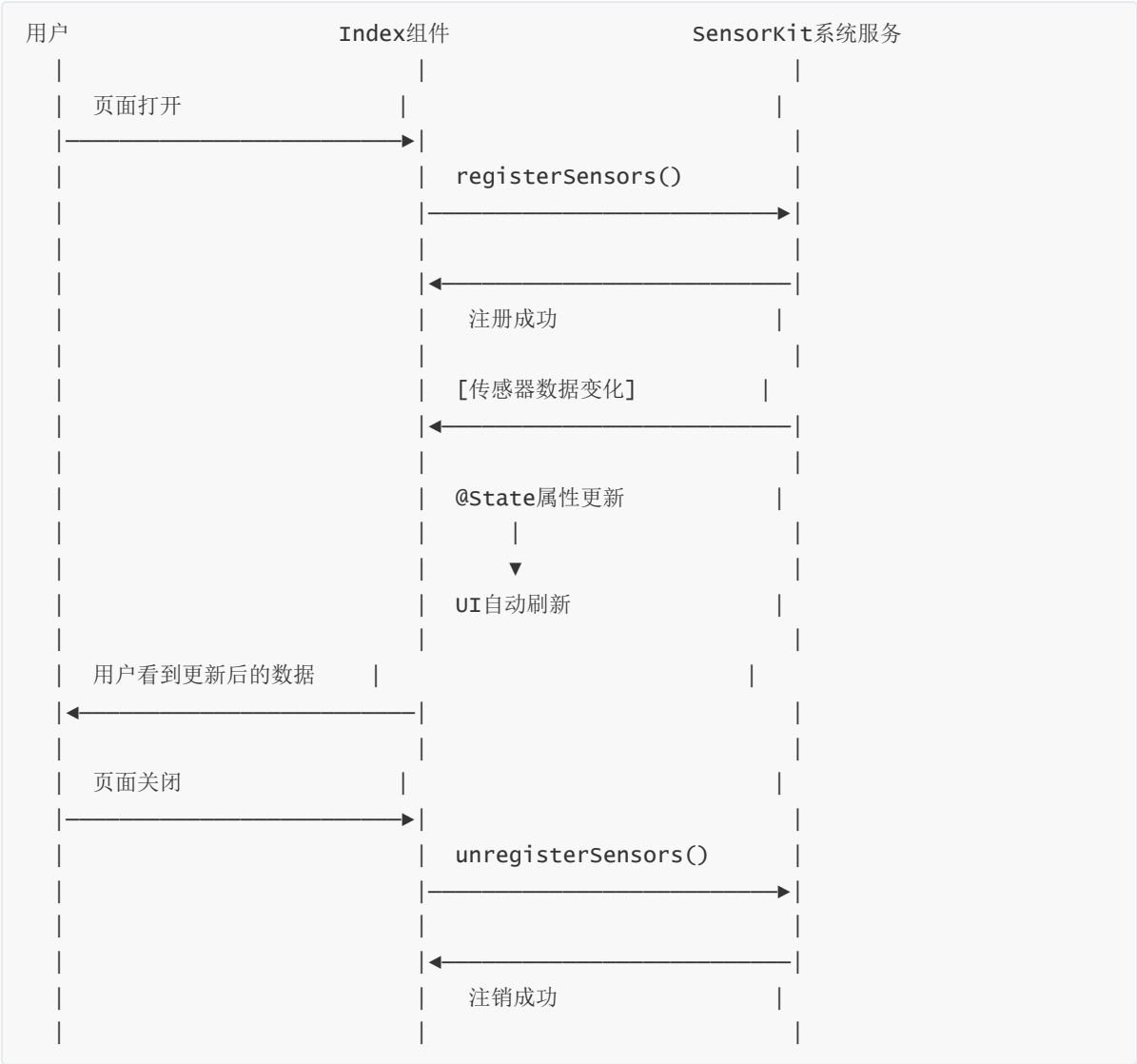
- **UI层**：采用ArkUI声明式组件，包含天气卡片、传感器卡片、详情卡片、同步卡片四个主要模块
- **业务逻辑层**：通过@State装饰器实现响应式数据绑定，管理传感器生命周期和同步状态
- **数据层**：存储传感器实时数据和模拟数据（用于传感器不可用时的降级处理）


```
|  
| }  
|
```

类图说明：

- **状态属性：**使用 `@State` 装饰器实现响应式数据绑定，属性变化自动触发UI刷新
- **方法：**包含数据处理方法（`getAqiLevel`、`getAqiColor`）、传感器管理方法（`registerSensors`、`unregisterSensors`）和同步模拟方法
- **构建器：**使用 `@Builder` 装饰器封装可复用的UI组件，提高代码复用性和可读性
- **生命周期：**`aboutToAppear` 初始化断点和注册传感器，`aboutToDisappear` 注销传感器避免内存泄漏

3.3 顺序图（传感器数据采集流程）

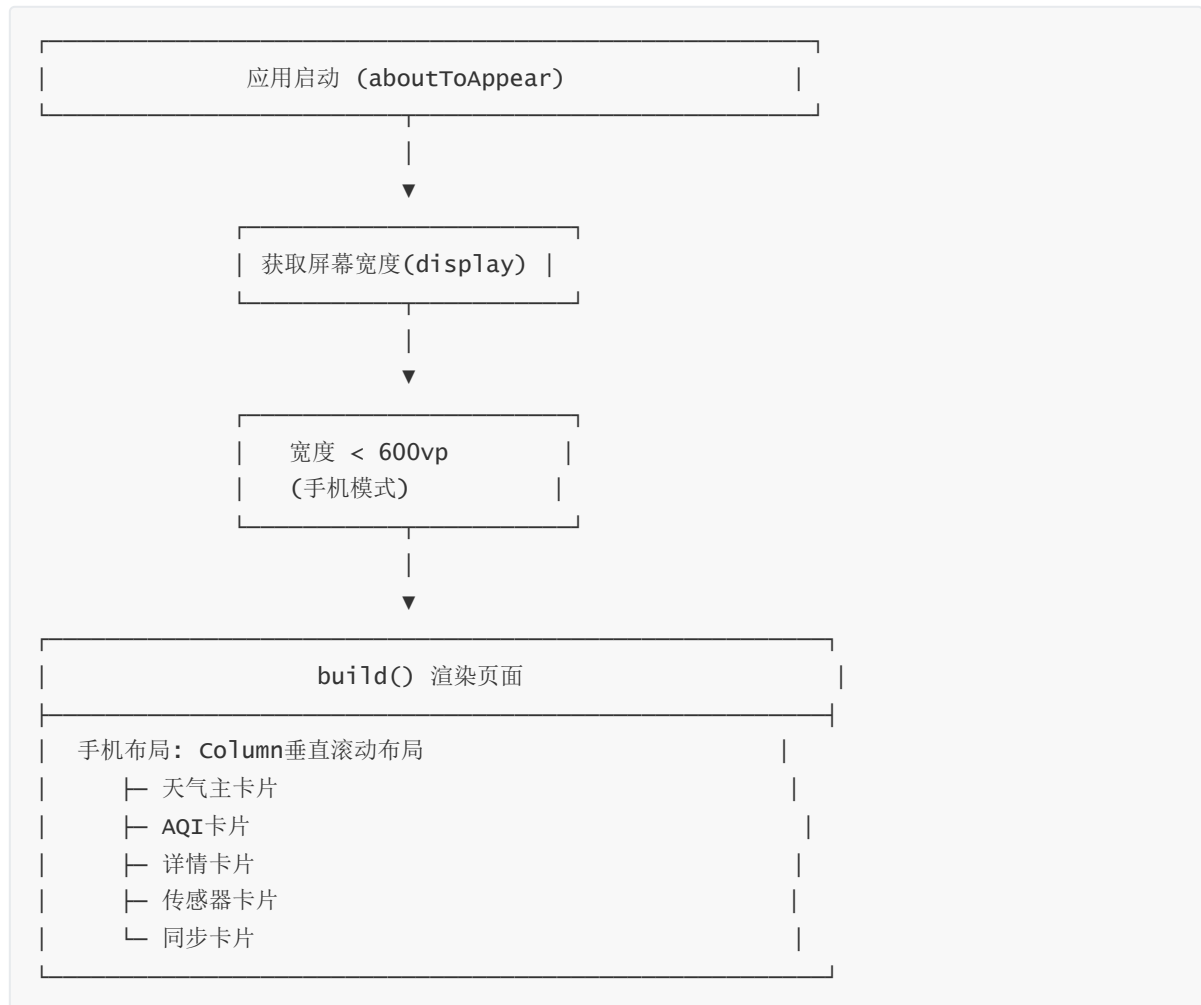


顺序图说明：

1. **页面打开：**用户打开天气应用，触发 `aboutToAppear` 生命周期
2. **注册传感器：**组件调用 `registerSensors()` 方法向SensorKit系统服务注册光线传感器和陀螺仪监听
3. **数据采集：**传感器数据变化时，系统服务主动回调数据到组件
4. **响应式更新：**`@State` 属性接收到新数据后自动触发UI刷新，用户看到实时数据

5. **页面关闭**：用户关闭页面，触发 `aboutToDisappear` 生命周期
6. **注销传感器**：组件调用 `unregisterSensors()` 方法释放传感器资源，避免内存泄漏

3.4 核心流程图（手机布局适配）



🔧 四、实现细节

4.1 核心代码分析

4.1.1 断点系统实现

```
initBreakpoint() {
  try {
    let displayInfo = display.getDefaultDisplaySync();
    let screenWidth = px2vp(displayInfo.width);
    if (screenWidth < 600) {
      this.currentBreakpoint = 'sm'; // 手机
    } else if (screenWidth < 840) {
      this.currentBreakpoint = 'md'; // 平板竖屏
    } else {
      this.currentBreakpoint = 'lg'; // 平板横屏
    }
  } catch (e) {
    this.currentBreakpoint = 'sm'; // 默认手机模式
  }
}
```

```
}
```

技术要点:

- 使用 `display.getDefaultDisplaySync()` 获取屏幕物理宽度
- 使用 `px2vp()` 将物理像素转换为虚拟像素, 确保不同分辨率设备的一致性
- 设置三个断点: `sm(<600vp)`、`md(600-840vp)`、`lg(>840vp)`
- 异常捕获确保即使API调用失败也能正常运行

4.1.2 传感器数据采集

```
registerSensors() {  
  // 光线传感器  
  try {  
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_AMBIENT_LIGHT,  
      (data: sensor.LightResponse) => {  
        this.lightIntensity = Math.round(data.intensity);  
      });  
    this.lightSensorAvailable = true;  
  } catch (e) {  
    this.lightSensorAvailable = false;  
    this.lightIntensity = 350; // 模拟数据  
  }  
  
  // 陀螺仪传感器  
  try {  
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE,  
      (data: sensor.GyroscopeResponse) => {  
        this.rotateX = Math.round(data.x * 100) / 100;  
        this.rotateY = Math.round(data.y * 100) / 100;  
        this.rotateZ = Math.round(data.z * 100) / 100;  
      });  
    this.gyroSensorAvailable = true;  
  } catch (e) {  
    this.gyroSensorAvailable = false;  
  }  
}
```

技术要点:

- 使用 `@kit.SensorServiceKit` 提供的传感器API
- 光线传感器类型为 `SENSOR_TYPE_ID_AMBIENT_LIGHT`
- 陀螺仪传感器类型为 `SENSOR_TYPE_ID_GYROSCOPE`
- 异常处理实现降级方案: 传感器不可用时使用模拟数据

4.1.3 多端布局适配

```
build() {  
  column() {  
    // 顶部标题栏  
    row() {  
      Text('智慧天气').fontSize(20).fontColor('#ffffff')    }  
  }  
}
```

```

Blank()
Text(this.currentBreakpoint === 'sm' ? '📱 手机' : '📺 平板')
}

// 根据断点选择布局
if (this.currentBreakpoint === 'sm') {
    this.phoneLayout()
} else {
    this.tabletLayout()
}
}
.linearGradient({
    direction: GradientDirection.Bottom,
    colors: [['#4facfe', 0.0], ['#00c6fb', 0.4], ['#f5f7fa', 0.85], ['#f0f0f0',
1.0]]
})
}

```

技术要点:

- 使用条件渲染实现布局切换
- 手机模式使用垂直滚动布局
- 平板模式使用双列网格布局
- 渐变背景增强视觉效果

4.2 资源配置

4.2.1 权限声明 (module.json5)

```

{
  "requestPermissions": [
    {
      "name": "ohos.permission.GYROSCOPE",
      "reason": "用于获取设备陀螺仪数据",
      "usedScene": {
        "abilities": ["EntryAbility"],
        "when": "inuse"
      }
    }
  ]
}

```

4.2.2 设备类型配置

```

{
  "deviceTypes": ["phone", "tablet"]
}

```

五、测试验证

5.1 测试环境

设备类型	模拟器型号	屏幕分辨率
手机	Pixel 5	1080x2340

5.2 测试用例

5.2.1 功能测试

测试项	测试步骤	预期结果	实际结果
天气信息展示	启动应用	显示合肥、22°C、晴、AQI 45	✅ 通过
手机布局	手机模拟器启动	单列垂直滚动布局	✅ 通过
光线传感器	模拟光照变化	实时显示lux值	✅ 通过
陀螺仪传感器	模拟设备旋转	实时显示三轴数据	✅ 通过
分布式同步	点击同步按钮	显示同步状态变化	✅ 通过

5.2.2 兼容性测试

测试项	测试步骤	预期结果	实际结果
传感器降级	关闭传感器权限	显示模拟数据	✅ 通过
异常处理	网络异常场景	应用正常运行	✅ 通过

5.3 测试截图说明

5.3.1 手机布局截图（sm模式）

17:59



282 KB/s

5G

4G

35%



琅琊区



请开启定位服务 >

PM
2.5

10

UV

弱



多云 13 ~ 26°C

24°C

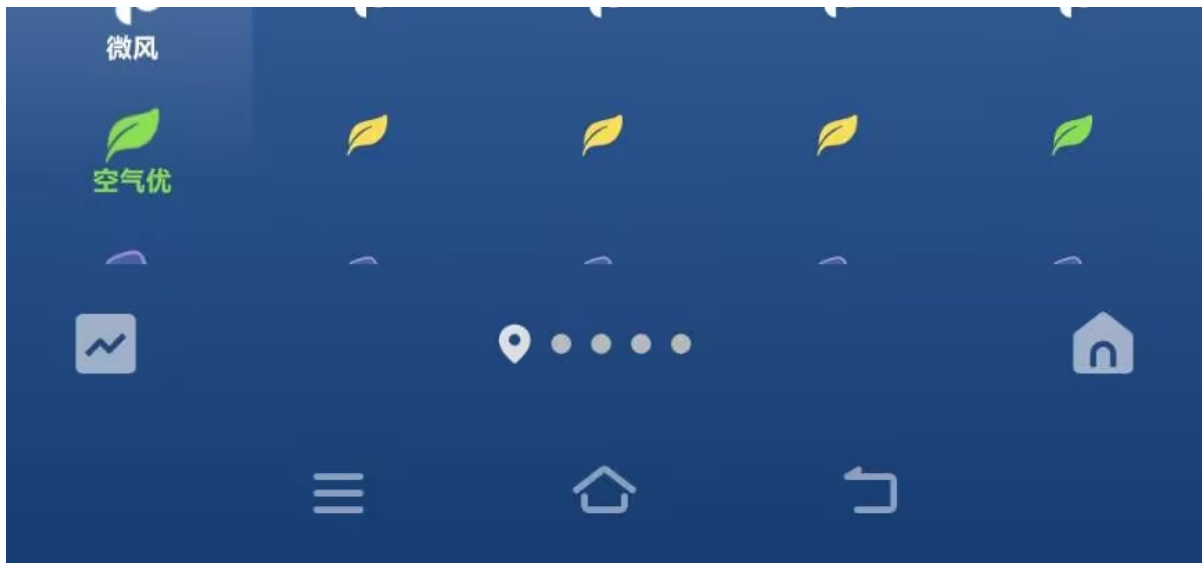
体感
22°C



微风轻拂，注意防护，保持舒适

24





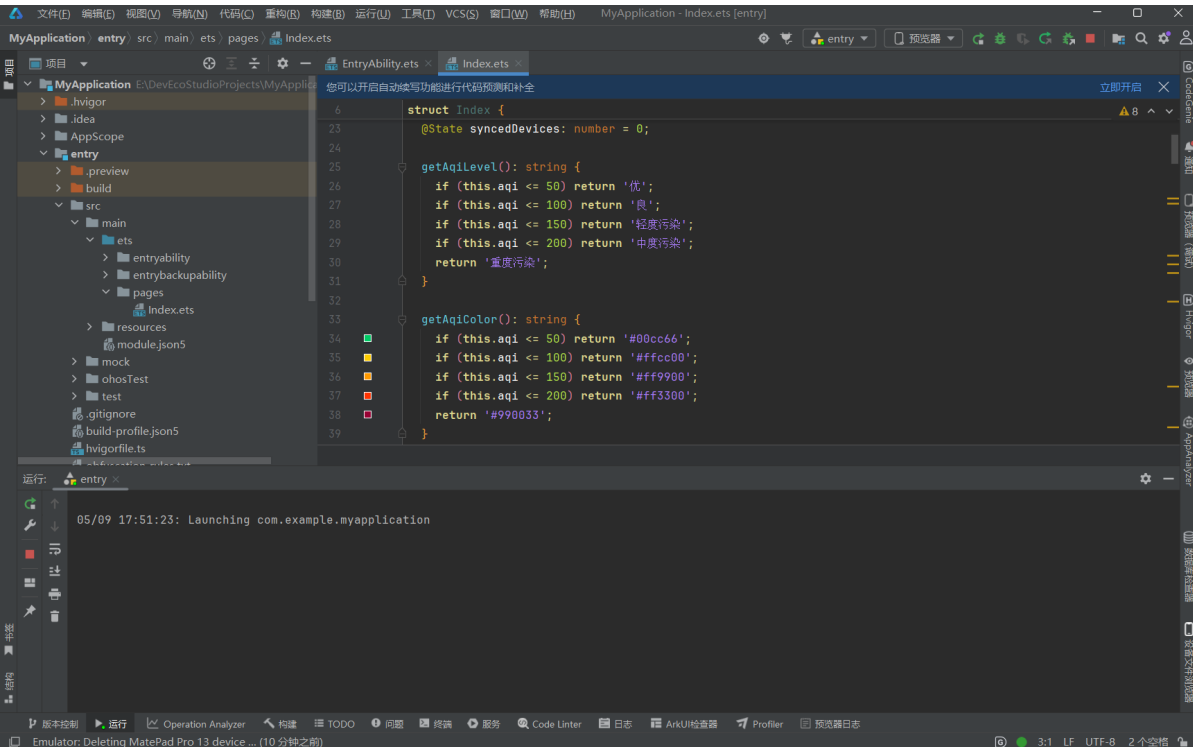
截图说明：

- **顶部标题栏：**显示"智慧天气"应用名称及" 📱 手机模式"标识
- **天气主卡片：**展示城市名称（琅琊区）、天气图标（☁️）、温度（24°C）、天气状况（多云）
- **AQI卡片：**显示空气质量指数（45）、等级（优）、湿度（65%）、风速（12 km/h）
- **详情卡片：**风向、体感温度、能见度、气压等详细信息
- **传感器卡片：**光线传感器和陀螺仪三轴数据
- **同步卡片：**分布式数据同步演示模块

布局特点：

- 单列垂直滚动布局，适合手机竖屏浏览
 - 支持弹性滚动，内容超出屏幕时可上下滑动
 - 卡片间距统一，视觉层次清晰
 - 渐变背景从蓝色过渡到白色，视觉效果舒适
-

5.3.2 测试环境展示截图



截图说明：

- **光线传感器**：实时显示环境光照强度（单位：lux）
- **陀螺仪传感器**：实时显示X/Y/Z三轴角速度（单位：rad/s）
- 传感器不可用时显示"(模拟数据)"标识，实现优雅降级

六、技术分析

6.1 手机布局技术

实现原理：

- 通过 `display.getDefaultDisplaySync()` 获取屏幕宽度
- 使用断点系统（sm）定义手机布局策略（宽度<600vp）
- 使用 `@Builder` 封装可复用组件，实现单列垂直布局
- 使用 `Scroll` 组件实现弹性滚动，支持内容超出屏幕时上下滑动
- 使用 `Column({ space: 16 })` 实现卡片间距统一

优势：

- 单列布局适合手机竖屏浏览
- 弹性滚动提供流畅的滑动体验
- 统一的设计语言和交互体验
- 卡片间距统一，视觉层次清晰

6.2 传感器集成技术

实现原理：

- 使用 `@kit.SensorServiceKit` 提供的统一传感器API
- 通过 `sensor.on()` 注册传感器监听回调
- 在 `aboutToAppear` 生命周期注册，`aboutToDisappear` 生命周期注销

注意事项：

- 必须声明相应的系统权限（如 `ohos.permission.GYROSCOPE`）
- 传感器可能不可用，需要实现降级方案
- 及时注销监听避免资源泄漏

6.3 分布式能力

演示方案：

- 通过模拟同步流程演示分布式数据管理原理
- 显示同步状态、已同步设备数、同步数据类型
- 提供同步和重置按钮交互

实际应用场景：

- 跨设备天气数据共享
- 智能家居设备联动
- 多端数据同步备份



七、总结与思考

7.1 实验总结

通过本次实验，我掌握了以下核心技能：

1. **ArkUI声明式开发**：理解了HarmonyOS NEXT的声明式编程范式，掌握 `@State`、`@Builder` 等核心装饰器的使用
2. **手机布局适配**：学会了基于断点系统的响应式设计方法，实现手机单列垂直滚动布局
3. **传感器数据采集**：掌握了光线传感器和陀螺仪的调用方法，理解了传感器生命周期管理
4. **分布式能力理解**：了解了HarmonyOS分布式数据管理的原理和应用场景

7.2 技术亮点

- **响应式设计**：通过屏幕宽度自动切换布局，提升用户体验
- **容错处理**：传感器不可用时自动使用模拟数据，保证应用可用性
- **代码复用**：使用 `@Builder` 封装组件，提高代码可维护性
- **生命周期管理**：正确注册和注销传感器，避免内存泄漏

7.3 改进方向

- 1. **动态数据**：接入真实天气API获取实时天气数据
- 2. **定位服务**：使用定位API自动获取当前城市
- 3. **深色模式**：实现深色/浅色主题切换
- 4. **数据持久化**：使用分布式数据库持久化天气数据
- 5. **动画效果**：添加传感器数据变化的动态效果

7.4 课后思考

1. 鸿蒙多端开发相比其他框架的优势？

HarmonyOS提供了统一的开发框架，一套代码可以运行在手机、平板、智能穿戴等多种设备上，具有天然的多端适配优势。相比React Native、Flutter等跨平台框架，鸿蒙多端开发更贴近原生体验，性能更好，同时提供了丰富的系统能力API。

2. 传感器在物联网中的应用场景？

传感器在物联网中有着广泛的应用，如智能家居中的环境监测（温度、湿度、光照）、智能穿戴设备中的运动追踪（陀螺仪、加速度计）、工业物联网中的设备状态监控等。传感器是物联网数据采集的核心，为智能化决策提供数据支撑。

3. 分布式技术对未来移动开发的影响？

分布式技术将改变传统移动应用的开发模式，实现多设备之间的数据共享和协同工作。未来的移动应用将不再局限于单一设备，而是可以跨设备无缝协作，为用户提供更加智能、便捷的体验。

八、附录

8.1 修改的文件列表

文件路径	修改内容
entry/src/main/ets/pages/Index.ets	完整的天气应用主页面
entry/src/main/module.json5	添加平板设备类型和陀螺仪权限
entry/src/main/resources/base/element/string.json	应用名称和权限说明字符串
entry/src/main/resources/base/element/color.json	启动窗口背景色
AppScope/resources/base/element/string.json	应用名称

8.2 核心依赖

依赖包	说明
@kit.SensorServiceKit	传感器服务API
@kit.ArkUI	UI框架（display模块）